Basic-Compreter für U 880

Dipl.-Ing. RAINER HEYNE

Mitteilung aus dem VEB Transformatoren- und Röntgenwerk „Hermann Matern" Dresden

Es wird ein komplexes Basic-Verarbeitungssystem vorgestellt, welches sich durch einen umfangreichen Befehlsvorrat, geringe Programmlaufzeiten und vor allem durch eine systematisch aufgebaute E-A-Schnittstelle und Generierungsfähigkeit auszeichnet. Es ist auf jedem Mikrorechnersystem mit dem Prozessor U 880 implementierbar. Der Speicherumfang beträgt 12 Kbyte für das Programm (ROM-fähig) und minimal 256 byte RAM für interne Arbeitszellen. Darüber hinaus ist je nach dem Umfang des Anwenderprogrammes ein weiterer RAM-Bereich erforderlich.

Für den ständig wachsenden Kreis der Anwender von Mikrorechnern ist die Erarbeitung der für den Betrieb notwendigen Programme zu einem zusätzlichen Problem geworden. Eine Sprache, die den Möglichkeiten der Mikrorechentechnik angepaßt ist und international schon sehr weite Verbreitung gefunden hat, ist Basic (Beginners All Purpose Symbolic Instruction Code). Basic wurde zu Beginn der sechziger Jahre vom Darthmouth College speziell für den Dialogbetrieb in Time-Sharing-Systemen für Kleinrechner entwickelt. Die hervorstechenden Merkmale sind die leichte Erlernbarkeit und die volle Dialogfähigkeit in allen Phasen der Programmerstellung, -testung und -abarbeitung.

Ein weiterer Vorteil des Einsatzes von problemorientierten Sprachen liegt in der Maschinenunabhängigkeit begründet. Eine Programmkompatibilität der Mikrorechner unterschiedlicher Konfigurationen und Betriebssysteme (Bürocomputer A 5110, A 5120, A 5130, Mikrorechnerentwicklungssysteme usw.) ist gegenwärtig unbefriedigend. Auf dem Niveau von Basic ist ein Austausch von Programmen aber nahezu uneingeschränkt möglich, falls ein einheitliches Verarbeitungssystem benutzt wird. Im folgenden soll daher ein leistungsfähiger Basic-Compreter vorgestellt werden.

Aufgabenstellung

Für den komplexen Einsatz der Dialogprogrammiersprache Basic standen bei der Implementierung folgende Forderungen:

- Alle Sprachelemente von Standard-Basic müssen unterstützt werden. Damit können die in der Literatur reichhaltig angebotenen Programme ohne weitere Entwicklungsarbeit genutzt werden.
- Darüber hinaus machen sich Spracherweiterungen in diesen Richtungen erforderlich:
  - Zeichenkettenmanipulation für die textorientierte Datenverarbeitung
  - Befehle zur Steuerung digitaler Prozesse wie Direktzugriff auf Speicher und E-A-Tore, logische Befehle, Maschinenprogrammaufrufe
  - umfangreiche E-A-Arbeit mit variabler Gerätewahl und einfache Dateiarbeit zur Beherrschung großer Datenmengen
- Minimierung der bei Interpretern typischen hohen Programmbearbeitungszeiten zur Steigerung der Effektivität
- leichte Anpaßbarkeit an die unterschiedlichen Gerätekonfigurationen und Betriebssysteme. Dazu müssen die typspezifischen Merkmale auf ein Minimum begrenzt und so konzentriert sein, daß sie schnell verändert werden können.

Arbeitsweise

Bei der Realisierung einer problemorientierten Programmiersprache gibt es zwei prinzipiell unterschiedliche Techniken, die Quellprogramme in eine der Hardware verständliche Form zu bringen. Man spricht von Compilern und Interpretern.

Compiler

Der Compiler ist ein Programmsystem, das das gesamte Quellprogramm geschlossen in ein Objektprogramm des Rechners überführt. Während des Übersetzungslaufes werden alle ausführbaren Anweisungen in Maschinenbefehle umgesetzt. Die nicht ausführbaren Anweisungen steuern die Arbeitsweise des Compilers (z. B. Speicherzuordnung, Strukturierung) bzw. ergeben im Objektprogramm Datenwerte (z. B. Konstanten, Bereiche).

Vorteile

Das einmal übersetzte Programm ist beliebig oft und ohne erneute Compilertätigkeit von der Hardware abarbeitungsfähig. Damit liegen der Speicherbedarf und die Laufzeit recht niedrig.

Nachteile

Werden Fehler im Programm festgestellt, beginnt der gesamte Objektprogrammerstellungszyklus wieder von vorn. Das Quellprogramm muß korrigiert werden (Editor), es muß komplett neu übersetzt werden (Compiler), und es muß, falls die Übersetzung fehlerfrei war, wieder geladen werden (Linker/Loader). Erst dann ist eine Fortsetzung des Programmtestes möglich.

Interpreter

Bei der Interpretertechnik wird das gesamte Quellprogramm in Speicher des Rechners abgelegt. Während der Ausführungsphase wird Quellzeile für Quellzeile analysiert und entsprechend den dort vorgefundenen Schlüsselworten und Anweisungen abgearbeitet. Soll eine Quellzeile mehrfach abgearbeitet werden, wie es bei Zyklen der Fall ist, so muß auch die Analyse jedesmal wieder durchgeführt werden. Zur Abarbeitung muß der Interpreter also noch im Rechner gespeichert sein.

Vorteile

Bei der Korrektur des Quelltextes kann das Programm sofort in seiner veränderten Form verarbeitet werden, da die Analyse erst zur Laufzeit erfolgt. Der Zeitaufwand für das Testen wird demnach niedrig liegen.

Nachteile

Die Analyse der Quellzeile erfolgt erst bei deren Aufruf und dann eventuell sehr oft. Damit steigt die Abarbeitungszeit des Anwenderprogrammes, da erheblicher Softwareaufwand erforderlich ist.

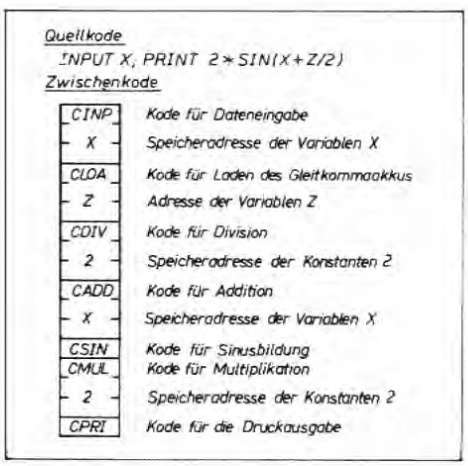
Ein derartiger Interpreter wurde in [3] vorgestellt.

Compreter

Durch eine Synthese beider Verfahren gelang es, deren sich scheinbar ausschließende Vorteile zu vereinen. Dabei wird in der Einschreibphase das Quellprogramm bereits analysiert. Werden Fehler festgestellt, so werden diese angezeigt und die Zeile nicht übernommen. Im fehlerfreien Fall wird für alle ausführbaren Anweisungen der Maschinen- oder ein spezieller Zwischenkode erzeugt (Compilerphase). Alle Angaben, die während des Einlesens noch nicht auswertbar sind bzw. bei Korrekturen verändert werden können, werden noch nicht in der Einlesephase verarbeitet, sondern erst zur Laufzeit interpretativ abgearbeitet (Interpreterphase). Im Ergebnis erhält man ein Programm, das fast so schnell wie beim Compiler abgearbeitet wird, gleichzeitig aber die Korrekturmöglichkeiten des Interpreters besitzt.

Bedingt durch die Arbeitsweise, nennt man international diese Übersetzungsprogramme Zwischenkodeinterpreter, compilierende Interpreter oder auch kurz Compreter.

Dieses letzte geschilderte Verfahren wird auch im erstellten Basic-System angewandt. Die Vorteile der Compretertechnik werden mit einem erhöhten Speicheraufwand erkauft. Wie im Bild an einer kurzen Anweisungsfolge demonstriert ist, unterscheiden sich der Quell- und der Zwischenkode wesentlich voneinander. Um aber die Dialogfähigkeit zu erhalten, ist ein Zugriff auf die eingegebene Quellinformation er-



Darstellung einer einfachen Anweisungsfolge im Quell- und Zwischenkode

forderlich. Deshalb erfolgt ihre Speicherung parallel zu der des Zwischenkodes.

Bei der Programmimplementierung gelangten noch eine Reihe weiterer spezieller Techniken zum Einsatz [1], die auch außerhalb des beschriebenen Systems interessante Lösungsmöglichkeiten für vielgestaltige Probleme bieten:

- Die Organisation aller vom Programm benutzten Daten erfolgt in einer Listenform. Durch den Einsatz der Listentechnik kann die gesamte Arbeit mit den unterschiedlichsten Daten standardisiert und durch eine geringe Anzahl von Standardunterprogrammen realisiert werden.
- Auf die Listentechnik aufbauend, wird die Verwaltung des Anwenderspeichers dynamisch vorgenommen. Damit wird es möglich, variabel lange Zeichenketten zu benutzen, die Grenzen von Feldern während des Programmlaufes zu verändern und eine beliebige Schachtelungstiefe von Unterprogrammen und For-Next-Zyklen zu erreichen.
- Die lexikalischen und syntaktischen Analysen erfolgen durch die Abarbeitung von lexikalischen bzw. Syntaxgraphen. Die Anwendung der Graphentheorie im Compilerbau führt zu einer sehr übersichtlichen Beschreibung der Syntax im Programmspeicher des Rechners und zu einer ausgezeichneten Änderungsfreundlichkeit.

Realisierter Sprachumfang

Eine Voraussetzung für die Implementierung des Systems war die Beherrschung aller im Standard-Basic vorhandenen Sprachkombinationen. Jedoch wurden darüber hinaus einige als sehr wesentlich angesehene Erweiterungen vorgenommen. Viele Anwender haben das bereits vorher getan. Leider hat sich aber international dazu noch kein Standard herausgebildet, und die zum Einsatz gekommenen Techniken sind sehr unterschiedlich. In der vorliegenden Version wurden Bemühungen unternommen, nicht noch eine weitere Variante zu entwerfen, sondern solche Erweiterungen auszusuchen, die am meisten Verbreitung gefunden haben und dem Charakter von Basic am besten entsprechen.

Einleitend werden einige allgemeine Eigenschaften der Sprache erläutert, anschließend ein Überblick über den gesamten Schlüsselwortvorrat gegeben.

Der Compreter beinhaltet eine Gleitkommaarithmetik für Zahlen mit einer Genauigkeit von neun Dezimalen und einem maximalen Exponenten von 38. Zusätzlich kann der Anwender auf eine 2-byte-Festkommarechnung (−32768...+32767) zurückgreifen. Diese einfache Zusatzarithmetik läßt sich durch den Prozessorbefehlssatz wesentlich einfacher realisieren und führt zu einer deutlichen Einsparung an Rechenzeit. Die Benutzung von Zeichenkettenkonstanten und -variablen ist gestattet. Alle Datenformen sind außer in der skalaren Form auch als Vektor oder Matrix definierbar.

In Anlehnung an höhere problemorientierte Sprachen ist der Gebrauch symbolischer Marken erlaubt. Im Standard-Basic kann als Sprungziel nur die Nummer der gewünschten Zeile angegeben werden. Hier kann außerdem jede Quellzeile durch eine oder mehrere symbolische Marken, bestehend aus maximal zwei Buchstaben, gekennzeichnet werden. Auf diese Marke wird durch Sprungbefehle des Programms zurückgegriffen. Diese Arbeitsweise führt zur größeren Übersichtlichkeit und leichteren Korrektur, da man sich von der konkreten Zeilennummer im Quellprogramm völlig lösen kann.

Die bedingte Sprunganweisung der Form

IF vergleich THEN zeilennummer

wurde zu einem leistungsfähigen Ausdrucksmittel weiterentwickelt. Erstens wurde sie durch die Aufnahme des ELSE-Zweiges zur vollständigen Alternative ausgebaut. Zweitens ist außer der Angabe von Zeilennummern die Abarbeitung kompletter Anweisungsfolgen hinter THEN und ELSE erlaubt.

Eine Aufzählung aller Schlüsselworte soll einen Überblick über den realisierten Befehlsvorrat geben. Während Standardelemente kommentarlos notiert werden, erfolgt für die weiterführenden Befehle eine kurze Erläuterung der Funktion.

Kommandos

BYE, LIST, LOAD, NEW, RENUMBER, RUN, SAVE

[NO]NUMBER	ohne/mit automatischer Zeilennumerierung
[NO]MEM	ohne/mit Quellprogrammspeicherung

Befehle

DATA, DEF, DIM, IF...THEN...[ELSE], FOR...TO...[STEP], GOSUB, GOTO, INPUT, LET, NEXT, ON, PRINT, READ, RESTORE, REM, RETURN, STOP
OPEN, CLOSE, GET,

PUT	erweiterte Dateiarbeit
ASGN	dynamische Gerätezuweisung
POKE, DOKE	Speicherzugriff schreiben 1, 2 byte
CALL	Aufruf Maschinenunterprogramm

Funktionen

ABS, ATN, COS, EXP, FNx, INT, LOG, RND, SGN, SIN, SQR

END, TOP, SIZE	dynamische Speicherverwaltung
RIGHT, LEFT, MID	Zeichenkettenmanipulation
LEN, VAL, STR, CHR, ASC	Konvertierung Zeichenkette ↔ numerischer Wert
PEEK, DEEK	Speicherzugriff lesen 1, 2 byte

Druckformatsteuerbefehle

BIN, HEX, FIX, FLO	Ausgabeformatanweisung
COL, SKIP, PAGE, TAB	Vorschubsteuerung

Operatoren

+, −, *, /, **, <, >, =, <=, >=, <>

.AND. , .OR. , .NOT. .NAND. , .NOR. , .XOR.	logische Verknüpfungen, 16 bit parallel
||	Zeichenverkettung

Mit diesem leistungsfähigen Befehlssatz ist es möglich, sowohl umfangreiche arithmetische Programme als auch solche zur Zeichenkettenverarbeitung und zur Steuerung von Prozessen zu gestalten. Einige größere Programme anderer Systeme aus der Literatur wurden bereits umgeschrieben und getestet. Fast alle voneinander abweichenden Sprachkonstruktionen ließen sich während der Einlesephase sofort ändern. In den meisten Fällen gelang es durch eine Ausnutzung der speziellen Besonderheiten des Sprachverarbeitungssystems (vollständige Alternative usw.), eine Programmverkürzung von 10...20 % zu erreichen (s. Tafel).

Abarbeitungszeiten einiger typischer Befehle

Befehl	ganze Zahlen	Gleitkommazahlen
Sprungbefehl	0,9 ms	
Transport (z. B. A = 1)	0,95 ms	1,1 ms
Addition	1,3 ms	1,8 ms
Multiplikation	1,9 ms	3,8 ms
Sinus	–	80 ms
FOR-NEXT-Organisation	2,2 ms	3,9 ms
Speicherdirektzugriff	0,9 ms	–

Implementierung im Anwendersystem

Beim gegenwärtigen Stand der Mikrorechentechnik fehlt auf Grund der unterschiedlichen Ausbaustufen noch eine verbindliche Vorgabe für die Gestaltung der gesamten Ein- und Ausgabearbeit. Dort treten erhebliche Unterschiede sowohl in der Hardware als auch in der Software der einzelnen Systeme auf. Diese betreffen mindestens die vom Anwender vergebenen Toradressen für die Geräte. Meist aber sind komplette Gerätesteuerungen stark voneinander abweichend. Die eingesetzten E-A-Geräte sind unterschiedlicher Bauart (vor allem Bildschirmsysteme und Tastaturen) und werden anders bedient (Polling oder Interrupt). Damit wird jede Anpassung von Programmen, die Ein- und Ausgabeoperationen beinhalten, wesentlich erschwert.

Um diesen Schwierigkeiten aus dem Weg zu gehen, wurde im Basic-System eine sehr allgemeine E-A-Schnittstelle geschaffen. Dabei wird es dem Anwender überlassen, seine Geräte durch spezielle, meist schon in seinem Betriebssystem vorhandene Peripheriesteuerprogramme zu bedienen. Es besteht prinzipiell die Möglichkeit, weiterhin vom Basic bisher nicht unterstützte Geräte anzusteuern. Das kann beim Anschluß von Zeichentischen oder Prozeßsteuerbaugruppen der Fall sein.

Zur Realisierung einer solchen Arbeitsweise wurden logische und physische Ein- und Ausgabe deutlich voneinander getrennt und alle physischen Prozeduren in einem gesonderten Programm zusammengefaßt. Nur dieses muß vom Anwender an seine speziellen Bedingungen angepaßt werden. Dessen Bestandteile sind für jedes unterstützte Gerät ein Steuerblock und ein bis drei Module.

Der Steuerblock enthält allgemeine Informationen über das Gerät. So beispielsweise die Richtung des Datentransfers (Ein-, Ausgabe), Informationen über das automatische Eröffnen und Schließen von Dateien usw. Außerdem sind die Start-

adressen der drei nachfolgend beschriebenen Module Bestandteil des Steuerblockes.

Der Modul „E-A-Routine" muß immer vorhanden sein und übernimmt den eigentlichen Datentransfer zwischen Basic-System und Gerät. Die Übergabe ist zeichenorientiert, das heißt, es wird generell ein Byte (im A-Register) übermittelt.
Der Modul „Open-Routine" kann vorhanden sein und hat die Voraussetzungen für den nachfolgenden Datentransfer zu schaffen. Das sind sowohl die Programmierung der betreffenden Peripheriebausteine als auch die speziellen Geräteinitialisierungen wie Löschen des Bildschirmes, Suchen einer Datei auf Folienspeicher bzw. Magnetbandkassette.
Der Modul „Close-Routine" ist ebenfalls wahlfrei. Falls vorhanden, wird er bei Auftreten der CLOSE-Anweisung oder entsprechend der Angabe des Steuerblockes bei Beendigung des Anwenderprogrammes aufgerufen und beschließt den Datentransfer vom oder zum Gerät. Beispiele hierfür sind abschließender Streifenvorschub beim Lochbandstanzer oder die Verzeichniseintragung beim Anlegen von Folienspeicherdateien.

Alle drei Module dürfen sämtliche Register verändern. Wird eine Routine fehlerhaft beendet, so ist das CY-Flag zu setzen. Darüber hinaus sind einige der insgesamt 15 Teilprogramme mit einem Generierungsteil ausgestattet, der dem Anwender alle Möglichkeiten der Speicheraufteilung, Bestimmung der Pufferlängen usw. bietet. Einem derartigen systematischen E-A-Aufbau wurde der Vorrang gegenüber den sonst in vielen Basic-Systemen üblichen Verbindung mit Sprungvektoren gegeben. Im vorliegenden Fall besitzt der Programmierer viel größeren Einfluß auf die Gestaltung der Treiberroutinen, die teilweise sicher schon für andere Programme ähnlich entworfen oder bereits Bestandteil des residenten Teiles des Betriebssystemes sind. Außerdem liefert er dem übergeordneten Basic die Voraussetzungen für eine sehr variable Dateiarbeit. So können alle Ein- und Ausgaben dynamisch im Programmlauf oder im Tischrechnermodus auf alle angeschlossenen Geräte zugewiesen werden. Beispielsweise ist es bei Druckerdefekten möglich, die Ausgabe auf Bildschirm oder Magnetbandkassette umzulegen.
Das Basic-System ist als Quellprogramm, versehen mit einer Sprachbeschreibung und einer ausführlichen Anwenderdokumentation, über den VEB Transformatoren- und Röntgenwerk nachnutzbar.

Literatur

[1] Heyne, R.: Entwicklung einer problemorientierten Programmiersprache mit entsprechendem Übersetzungsprogramm für den K 1520. Diplomarbeit an der IH Dresden, 1983
[2] Hoffmann, P.: Eine Entwurfsstrategie für Diagonalsysteme – Teilnehmersteuerung und inkrementelle Kompilierung am Beispiel von BASIC-4201. Dissertation an der IH Dresden, 1979
[3] Lübcke, J.; Villbrandt, R.: BASIC-Interpreter für K 1520. radio fernsehen elektronik, Berlin 32 (1983) 1, S. 14–16

Low-Power-Schottky-TTL DL 000

Dr.-Ing. ECKHARD FEHSE

Mitteilung aus dem VEB Halbleiterwerk Frankfurt (Oder)

Low-Power-Schottky-TTL-IS der zweiten TTL-Generation aus dem VEB Halbleiterwerk Frankfurt (Oder) zeichnen sich durch eine drastische Reduzierung ihrer Verlustleistung aus. Obwohl Kompatibilität zur D-10-Serie besteht, weichen doch einzelne Kenngrößen und Einsatzbedingungen voneinander ab. Dies ist neben einer Auflistung des vorgesehenen Typenspektrums Gegenstand des nachfolgenden Beitrags.

Die Low-Power-Schottky-Technologie ermöglicht gegenüber der Standard-TTL eine Verbesserung des Geschwindigkeits-Leistungsproduktes um den Faktor 5. Bei gleicher Gatterverzögerungszeit wird der Leistungsverbrauch um 80 % gesenkt. Diese Tatsache zieht weitgehende erfreuliche Konsequenzen für den Geräteentwickler nach sich:

- Senkung der Verlustleistung bei konstanter Packungsdichte
- Erhöhung der Zuverlässigkeit
- Verkleinerung der Netzteile
- kleinere Stromdichten und damit weniger Störungen.

Der geringere Eingangsstrom ermöglicht eine bessere Schnittstellengestaltung zu MOS-IS. Ab einer Frequenz von etwa 5 MHz ist der Leistungsverbrauch von LS-TTL-Schaltkreisen kleiner als der der entsprechenden CMOS-IS.
Basis der Schottky-Technologie ist der durch eine Schottkydiode geklemmte Transistor (s. Bild 1). Die Schottky-Diode hat gegenüber einer normalen Siliziumdiode zwei entscheidende Vorteile:

- sehr hohe Geschwindigkeit auf Grund der kurzen Sperrerholzeit
- kleinere Durchlaßspannung bei gleichem Strom.

Sie verhindert, daß der Transistor zu sehr in die Sättigung gerät und ermöglicht damit das Sperren in wesentlich kürzerer Zeit.
Die Kennzeichnung der neuen Low-Power-Schottky-Baureihe des HFO erfolgt durch die Buchstaben DL... Die nachfolgenden

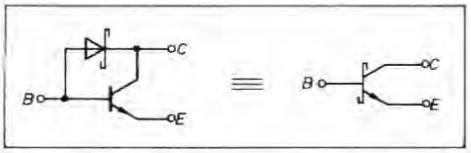


Bild 1: Durch Schottky-Diode geklemmter Transistor

drei Ziffern geben Auskunft über die logische Funktion. Es besteht, abweichend zur D-10-Serie, volle Übereinstimmung mit der international üblichen 74er Nomenklatur.

Typenspektrum und Kenngrößen

Die Entwicklung der Low-Power-Schottky-IS erfolgt in mehreren Abschnitten. Im ersten Abschnitt werden die in Tafel 1 aufgeführten Typen realisiert.
Für diese IS sind in den Tafeln 2, 3 und 4 die elektrischen Kenngrößen aufgeführt.
Entsprechend der Tafel 5 wird diese Reihe erweitert.

Zusammenschaltungsbedingungen

Obwohl die Low-Power-Schottky-TTL mit den anderen Schaltkreisfamilien kompatibel ist, gibt es doch einige Besonderheiten und Abweichungen. Im Gegensatz zu den meisten IS der D-10-Serie befinden sich an den Eingängen Clamping-Dioden. Sie sollen nur negative Störspitzen von den Eingängen fernhalten und sind nicht für permanente Belastung ausgelegt. Neben den bei anderen Schaltkreisfamilien gebräuchlichen Multiemittereingängen (Bild 2) kommen in der Low-Power-Schottky-Technik noch drei andere Eingangsschaltungen zur Anwendung (siehe Bilder 3 bis 5). Sie zeichnen sich neben der höheren zulässigen Eingangsspannung von 7 V besonders durch geringere Ein- und Ausgangsströme aus (Tafel 6). Letzteres macht eine Verdopplung des Ausgangslastfaktors von 10 auf 20 möglich. Zu beachten ist weiterhin, daß der maximale Ausgangs-Low-Strom bei Einhaltung des L-Pegels von 0,5 V auf 8 mA begrenzt ist. Auf Grund der für Low-Power-Schottky-TTL zulässigen Ein- und Ausgangsströme ergeben sich die in der Tafel 7 aufgeführten Zusammenschaltungsmöglichkeiten, wobei die maximalen Eingangsströme eines Diodeneinganges zugrunde gelegt wurden.

Zur Erhöhung des Lastfaktors können Ausgänge von Low-Power-Schottky-TTL-Gattern parallelgeschaltet werden, wenn durch die Ansteuerung ein gleicher logischer Zustand gewährleistet ist. Jedoch sollten nur Gatter, die sich auf dem selben Chip befinden, zusammengeschaltet werden. Beim Parallelschalten von Gattern, die sich auf verschiedenen Chips befinden, führen die möglichen Unterschiede in den Gatterlaufzeiten zu Ausgleichsströmen, welche nicht so sehr die Endstufen belasten, sondern logische Probleme in sich bergen, wenn die Gatter z. B. als Takttreiber benutzt werden.

Im Gegensatz zu Standard-TTL-IS können IS der Low-Power-Schottky-TTL-Reihe direkt von CMOS-IS angesteuert werden, da ihr Eingangs-L-Strom kleiner als 360 µA ist. Beim Ansteuern von CMOS-IS muß ein Pull-up-Widerstand eingesetzt werden, um den CMOS-H-Pegel sicher zu erreichen.